Project Sprint #2

The Solitaire game is described in CS449HomeworkOverview.docx. You should read the description very carefully.

Your submission must include the GitHub link to your project, and you must ensure that the instructor has the proper access to your project. You will receive no points otherwise.

GitHub link: https://github.com/jecalvil/sprint_2

In this assignment, you aim to implement the minimum features including **choosing the board size and type, starting a new game**, **making a move, determining if a game is over**. The following is a sample GUI layout.

The implementation of a GUI is required. You should make every attempt to make your code easy to extend. It is required to separate the user interface code and the game logic code into different classes (refer to the TicTacToe example). xUnit tests are required.

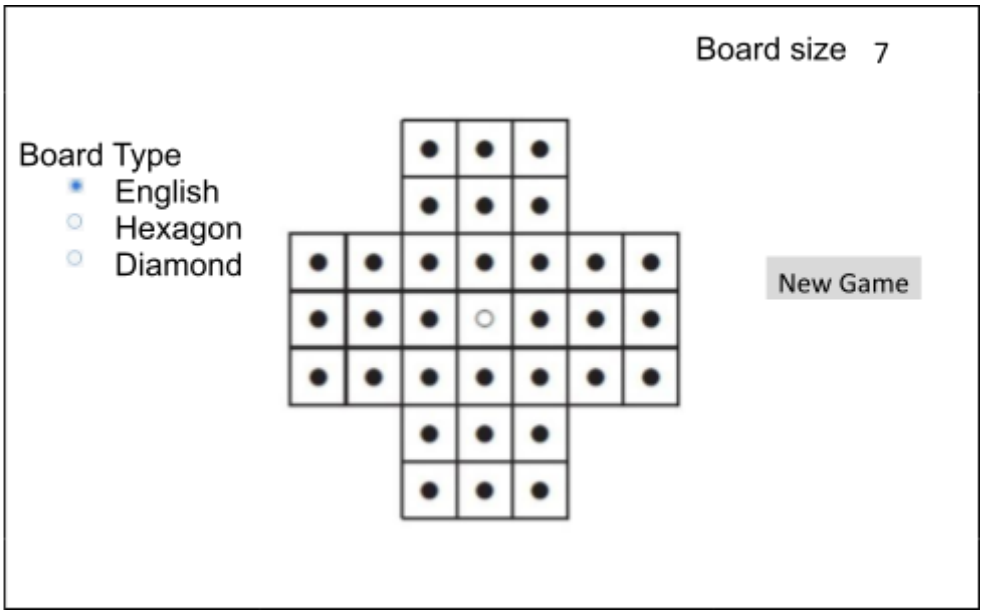


Figure 1. Sample GUI layout of the Sprint 2 program

Deliverables:

1. **Demonstration (8 points)**

   Submit a link to a video of no more than three minutes, clearly demonstrating that you have implemented the required features and written some automated unit tests. In the video, you must explain what is being demonstrated. No points will be given without a video link.
   YouTube/Panopto link:
   https://umsystem.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=a89faccb-803d-4f3d-ab77-b4f000394ec0

	Feature

1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game
5	Automated unit tests

2. Summary of Source Code (1 points)

Source code file name	Production code or test code?	# lines of code
board.py	Production	215
components.py	Production	52
peg_solitaire_gui.py	Production	405
test_peg_solitaire.py	Production	414
PressStart2P-Regular.ttf	Other (Font File)	N/A
	Total	1086

You must submit all source code to get any credit for this assignment.

3. Production Code vs User stories/Acceptance Criteria (3 points)

Update your user stories and acceptance criteria from the previous assignment (if required) and ensure they adequately capture the requirements. Summarize how each of the following user story/acceptance criteria is implemented in your production code (class name and method name etc.). If you are not using an OOP, then adapt the tables accordingly.
Note: My User Stories had different names and slightly different users IDs (5 instead of 4, so I changed this table to match)

User Story ID	User Story
1	Choose board size
2	Choose the board type
3	Start a new game of the chosen board size and type
4	Make a move in a game
5	A game is over

User Story ID and Name	AC ID	Class Name(s)	Method Name(s)	Status (complete or not)	Notes (optional)
1 Choose a board size	1.1	Board, PegSolitaireApp, RadioButton	Board.is_valid_size, Board.__init__ options_screen, Radiobutton.draw, Radiobutton.check	Complete	1.1 minor change, instead of inputting a number, it is selected as it uses radio buttons instead of text input. (User Story 1 tests are

					automatic and manual)
	1.2	Board, PegSolitaireApp, RadioButton	Board.is_valid_size, Board.__init__ options_screen, RadioButton.draw, RadioButton.check	Complete	1.2 large story change, no more error message for invalid size as radio buttons don't make it possible so it is unnecessary, but it is tested in code format to see if we made one with a size 6 for example, and manual checks in the gui
	1.3	Board, PegSolitaireApp, RadioButton	Board.is_valid_size, Board.__init__ options_screen, RadioButton.draw, RadioButton.check	Complete	1.3 Similar to 1.2, it has radio buttons instead of text input so no more error message, but there are checks for misinputs at the code level that are out of range, or symbols, user story updated to fit
2 Choose the board type	2.1 (default board type)	PegSolitaireApp, RadioButton	Options_screen, RadioButton.draw, RadioButton.click	Complete	Tested via GUI
	2.2 (successful board type selection)	PegSolitaireApp, RadioButton	Options_screen, RadioButton.draw, RadioButton.click	Complete	Tested via GUI (manual test)
	2.3 (only one selected at a time)	PegSolitaireApp, RadioButton	Options_screen, RadioButton.draw, RadioButton.click	Complete	Tested via GUI (manual test)
3. Start a new game of the chosen board	3. 1	Board, PegSolitaireApp	Board.__init__, is_english_corner, game_screen	Complete	None
	3.2	Board, PegSolitaireApp	Board.__init__, is_diamond_corner,	Complete	None

			is_hexagon_corner, game_screen		
	3.3 (will be removed)	Board, PegSolitaireApp, Radio_Button	Board.__init__, is_valid_size, RadioButton.draw/click	Complete (completed as it falls under other criteria, but will be removed, it is simply kept for now as note to self and grader)	This acceptance criteria will be removed due to the fact the program uses radio buttons now instead of text, it is too redundant as it falls to similar to the checks in stories 1 and 2
4. Make a move in a game	4.1	Board, PegSolitiareApp	get_legal_moves, is_valid_move, game_screen (to show highlighted moves)	Complete	Automatic and Manual test, automatic to verify legal move and manual to see legal move highlighted
	4.2	Board, PegSolitiareApp	Make_move, game_screen	Complete	Mix of automatic and manual, automatic to check validity and change, manual for visual aspects Criteria changed to also mention both orthogonal and diagonal moves
	4.3	Board, PegSolitiareApp	Make_move, is_valid_move, game_screen (to show error message)	Complete	Mix of automatic and manual, by making sure the board stays the same logically and that corresponding highlights show and pegs don't change. Acceptance criteria changed for the peg to become unselected after invalid move instead of remaining selectede
	4.4	Board, PegSolitiareApp	Get_legal_moves, game_screen (show error)	Complete	Mix of automatic and manual, automatic tests board logic and manual tests visual
	4.5	PegSolitaireApp	Game_screen (selection/deselection)	Complete	Manual Test
	4.6	PegSolitaireApp	Game_screen (selection/deselection when clicking another peg)	Complete	Manual Test

5 - A game is over	5.1	Board, PegSolitiareApp	Game_over_check, count_pegs, get_rating, game_screen	Complete	Modified Acceptance Criteria to be specific about how to make a new game. It is now either done using the restart button for the same settings or by clicking back for different settings. Mix of automatic and manual, to check if game is over on the logic side and that visual rating shows
	5.2	Board, PegSolitaireApp	Check_win, get_rating, game_screen	Complete	Automatic side: tests board logic to check win, Manual: visually show win screen
	5.3	Board	game_over_check	Complete	Just an automatic test to see if game is not over
	5.4	PegSolitaireApp, MenuButton	Game_screen,	Complete	Removed acceptance criteria for being able to create a new game as that requirement is checked under 5.5 to avoid redundancy
	5.5	PegSolitaireApp, MenuButton	Game_screen, MenuButton.check_click	Complete	Adjusted acceptance criteria to specify a new game is made by pressing restart or by going back to the settings menu

4. Tests vs User stories/Acceptance Criteria (3 points)

Summarize how each of the user story/acceptance criteria is tested by your test code (class name and method name) or manually performed tests.

User Story ID	User Story

1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game

4.1 Automated tests directly corresponding to the acceptance criteria of the above user stories

You are required to use ChatGPT to create at least 2 unit tests. You also need to ensure that the generated test cases are correct and refine them if not. At the end of the submission, provide the screenshots of your ChatGPT prompts and answers, along with errors ChatGPT made and you fixed. You may also use another LLM, including hosted locally. Points will be deducted if no screenshots are provided.

Note: Unit tests made by ChatGPT are the ones in Story 4 and 5, they are highlighted.

User Story ID and Name	**Acceptance Criterion ID**	**Class Name (s) of the Test Code**	**Method Name(s) of the Test Code**	**Status (Pass/Fail/In Progress)**
1 Choose board size	1.1	TestBoardSize	Test_valid_board_size test_valid_sizes_accepted_as_text	Pass
	1.2	TestBoardSize	Test_invalid_board test_other_even_sizes_rejected	Pass
	1.3	TestBoardSize	Test_out_of_range_rejected Test_non_numeric_rejected test_valid_sizes_accepted_as_text test_unknown_board_type_rejected	Pass
2 Choose a board type	2.1, 2.3, 2.3	N/A - tested manually	N/A	Pass (Manual)
3. Start a new game of the chosen board size and type	3.1	TestNewGame	Test_english_7_counts test_english_other_sizes	Pass
	3.2	TestNewGame	Test_diamond_counts Test_hexagon_counts test_center_only_empty	Pass

	3.3 (AC will be removed, but a test exists that is similar, more of a code exclusive level test)	TestNewGame	test_invalid_size_falls_back_to_default	Pass
4. Make a move in a game (Highlighted ones made with chatGPT)	4.1	TestMakeMove	test_ac_4_1_peg_has_legal_moves	Pass
	4.2	TestMakeMove	test_ac_4_2_valid_jjump_updates_board	Pass
	4.3	TestMakeMove	test_ac_4_3_illegal_move_leaves_unchanged	Pass
	4.4	TestMakeMove	test_ac_4_4_empty_or_immobile_position_has_no_legal_moves	Pass
5. A game is over	5.1	TestGameOver	Test_ac_5_1_no_legal_moves_remain	Pass
	5.2	TestGameOver	test_ac_5_2_one_peg_remaining	Pass
	5.3	TestGameOver	test_ac_5_3_game_continues_when_legal_moves_remain	Pass

4.2 Manual tests directly corresponding to the acceptance criteria of the above user stories (if any)

User Story ID and Name	**Acceptance Criterion ID**	**Test Case Input**	**Test Oracle (Expected Output)**	**Notes**
1 Choose board size	1.1, 1.2, 1.3	Launch game to setup screen and inspect board size section	Only odd radio buttons 5, 7, and 9 show with 7 selected as default	Pass via GUI inspection
2 Choose board type	2.1	Launch game and view setup screen	Three board types are presented (English, Diamond, hexagon) with English selected by default	Pass via GUI inspection
	2.2	Click different radio button (e.g Hexagon or Diamond) while english is selected	Hexagon or Diamond (corresponding click) becomes selected while English becomes unchecked, screen made with proper board type chosen	Pass via Gui inspection

	2.3	Click another board type while one is already selected	Only one board type remains selected at all times and board size configuration remains unaffected	Verified via GUI
4 - Make a move in a game	4.1	Click peg that has at least one legal jump available	Hole that the peg can jump to is highlighted	Verified via GUI
	4.2	Make a valid jump	Selected peg moves into empty hole, peg it jumped over becomes empty	Verified via GUI
	4.3	Selecting a peg with legal moves and trying to make an illegal jump	Peg does not move, board state stays the same, selected peg removes highlight, error message displays move not allowed	Verified via GUI
	4.3	Selecting a a peg with no legal moves	Message displays saying that a move is not available from that position, nothing is selected, board state remains the same	Verified via GUI
	4.5	Selecting a peg, then clicking it again	Peg removes its initial highlight becoming deselected, board state remains the same	Verified via GUI
	4.6	Selecting a peg with legal moves, then selecting another peg with legal moves	New peg is highlighted with legal moves and old peg has highlights and legal move highlights removed	Verified via GUI
5 - A game is over	5.1	Game is played to where the game is not won (more than one peg remains)	Message indicating Game Over, message with performance (marbles left), can create a new game via back or restart button	Verified via GUI
	5.2	Game is played to where the game is won (only one peg remaining)	Message indicating game was won, message showing performance and peg count of 1/Outstanding, game can be created via back button or restart button	Verified via GUI
	5.4	Game is played to where the game is over (either a win	No remaining pegs can be selected, no move can be made,	Verified via GUI

		or a loss), i.e no peg moves remain	board state stays the same as when it ended, message showing corresponding game rating	
	5.5	Game is played to where the game is over (either a win or a loss), i.e no peg moves remain	A new game can be created either by going back and starting game or via the restart button for the same game mode, and the game over/win message is cleared	Verified via GUI

4.3 Other automated or manual tests not corresponding to the acceptance criteria of the above user stories (if any)

Number	**Test Input**	**Expected Result**	**Class Name of the Test Code**	**Method Name of the Test Code**
Aligns with Story 5, but with not any specific acceptance criteria	A premade solution to an English size 7 game	Winning game with outstanding rating	TestWinningPlay Through	test_english_7_full _winning_game

I decided to make an automated test using Claude, that falls under Story 5 of a winning game. It overall encompasses all of it and not any particular acceptance criteria, so I put it in the other category. The screenshots for the prompt and output are also provided below.

LLM evidence: Screenshots for the prompts and results will be below, as they are very long I placed them at the bottom.

Explanation how I checked them and corrected for any errors:

Firstly, I needed to decide which sections I wanted to use in chatGPT to create the code for the unit tests. I decided to make the code for the tests that correspond to the AI created User stories/Acceptance Criteria from Sprint 1, in order to maintain the similarity. Afterwards, to try to keep it as accurate as possible, I uploaded my previous user stories from sprint 1 as well as its acceptance criteria, along with the sprint 2 doc just in case my prompt didn't give it initial context. I also supplied it with the code I had developed so far in order for the LLM to understand what I was building and what I had, so it would better understand my structure and not make code that was so dissimilar that I would have to make massive changes to the code I already had, such as perhaps its overall architecture for the unit tests were too different to my logical implementation, requiring me to make more drastic differences. I also informed it of some of the changes I made and the direction of the overall project, specifically mentioning my change using radio buttons for some of the acceptance criteria and having it focus on creating unit tests that are best suited for automated testing. I also copied and pasted some helper functions that I had, so it could use it to produce cleaner unit tests in case it wanted to use them.

After I had gotten the results, I checked its explanation and I overall agreed with them. I inspected the code and I first noticed that I needed to not reuse the imports, as they had theirs created to where each main story would have its own file for unit testing. Afterwards, I looked at the functions to see if it met the requirements based on the acceptance criteria. I felt that the first initial attempt captured the overall essence, but it missed a lot of the details. For example, in the initial code that it had given me, it did give tests for the overall essence, such as testing if the game continues when legal moves exist as the first method, but it didn't capture every single acceptance criteria, such as the very first one, where it didn't really do a check to see if a selected peg was a legal move. It didn't have the approximate one to one correspondence to each of the AC ids that I was trying to go for. One could argue that it did mean the criteria overall as the tests combined some of them, but it was better to keep them separate and corresponding to the AC ids, as we want to be able to isolate and easily check each of them. Overall, this made it difficult to check for the correctness as everything overlapped/was tightly coupled. Therefore, I prompted again to make a method to correspond to each of the acceptance criteria numbers, in this way it would be more organized and I would be able to easily see whether each of the criteria were met.
This time, although I didn't quite like the name scheme for the method names, they were more alike to each of the acceptance criteria so they were easier to check. No since it was more organized, I went 1 by 1 with each of the acceptance criteria with its corresponding method and checked whether the logic it had placed made sense.

For example, the very first one, I wanted to check if a selected peg had a legal move, which it didn't explicitly include before. Here it created a custom board with 2 pegs that were right next to each other (4,2) and (4,3). According to our rules, a move is 2 spaces away that is either diagonal or orthogonal. Therefore, it checked what would be a legal move, which I verified which would be (4,4), and checked whether that was in the list for legal moves when calculating the legal move function.

I also attempted to check for completion, meaning whether the provided test captured all the different points within each of the acceptance criteria and modified and added accordingly. One particular instance was for 4.2 where it makes a valid jump. Overall, the test case it provided was fairly accurate, as the pieces it placed on the board and its move made logical sense, but it didn't capture the whole criteria, by checking both orthogonal and diagonal moves. So in order to fix it, I made sure to continue to have the orthogonal check, but then with the same setup, created a scenario that did a diagonal move. In this way, I was able to test and capture what it meant to make a legal move, instead of just a particular instance.

I also gave it freely fair reign to check which ones it could more accurately represent with automated tests vs ones where manual tests may be better. One particular example is with the functions for 4.4 and 4.5. Within these, the tests were more visual as they were about selecting and deselecting and showing highlighted pegs and move positions. Here it was correct to perform the @unittest.skip method, by simply describing the acceptance criteria by noting it down, to then leave me to perform the manual verification afterwards.

This process was overall very similar to story 5. It created tests corresponding to each of the acceptance criteria and left ones that were meant to be more manual, noted with the @unittest.skip method. I also then looked at the coordinates it picked to set up the scenario and checked whether the action it was going to perform and the placements of the pegs made sense. All of this was overall very accurate, similar to the one before.

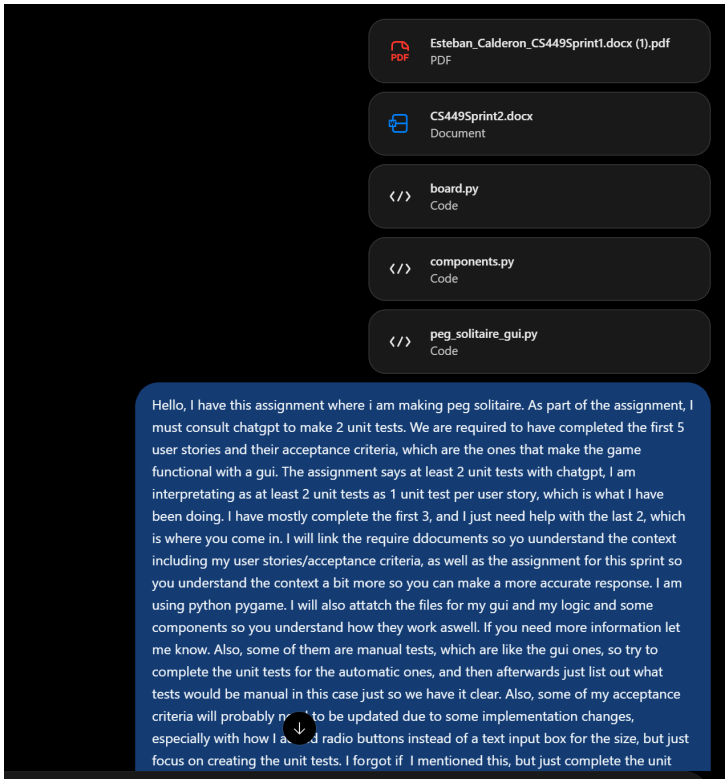
To conclude the usage of the LLM in this case, it was overall very helpful and quick to make the unit tests and have them correspond to the acceptance criteria. The main issue the LLM faced was having proper coverage of all the requirements, or as one could say it struggled understanding the "Definition of Done". This then had to be checked over to make sure it overall covered the acceptance criteria and that the logical placements made sense. The styling for the programming was also a little bit poor as some of the namings and contents don't feel

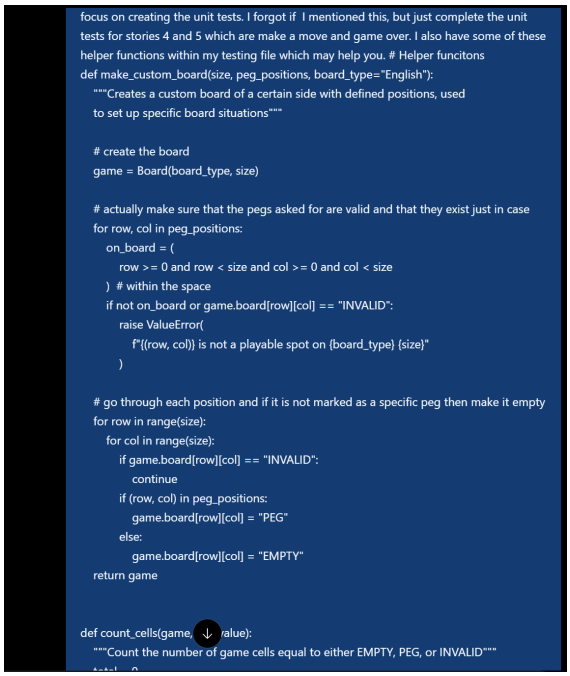
very user friendly, but they were kept as after some time one could understand them and they did feel logically correct. The LLM did not face any programming or syntax errors; It didn't struggle with that at all. The main problem was coverage and understanding the scope of the project which is where I had to perform my fixes and interventions.

Another thing to note is that many of these functions weren't completely wrong either, as some may need to add more test cases to be more thorough. For example, I just have one test case for the orthogonal and diagonal move which I had to fix by adding that test case. It could be more thorough by creating several more instances by testing all the positions, but I felt that it wasn't necessary. Perhaps in a setting where more thorough testing is critical, one may want to prompt the AI to make the tests more thorough, but I felt that for a class project game, it was thorough enough.

Afterwards, I tested with Claude to see if it could make and run a premade solution. I chose Claude as through my experience it is the "smartest". I then ran through the provided solution to see if it made sense while play testing it and it was valid. I wanted to keep a full premade solution just in case something were to happen in the future, so I can thoroughly check a full playthrough quickly

Prompts and Outputs:





```
focus on creating the unit tests. I forgot if  I mentioned this, but just complete the unit
tests for stories 4 and 5 which are make a move and game over. I also have some of these
helper functions within my testing file which may help you. # Helper funcitons
def make_custom_board(size, peg_positions, board_type="English"):
    """Creates a custom board of a certain side with defined positions, used
    to set up specific board situations"""

    # create the board
    game = Board(board_type, size)

    # actually make sure that the pegs asked for are valid and that they exist just in case
    for row, col in peg_positions:
        on_board = (
            row >= 0 and row < size and col >= 0 and col < size
        )  # within the space
        if not on_board or game.board[row][col] == "INVALID":
            raise ValueError(
                f"{(row, col)} is not a playable spot on {board_type} {size}"
            )

    # go through each position and if it is not marked as a specific peg then make it empty
    for row in range(size):
        for col in range(size):
            if game.board[row][col] == "INVALID":
                continue
            if (row, col) in peg_positions:
                game.board[row][col] = "PEG"
            else:
                game.board[row][col] = "EMPTY"
    return game


def count_cells(game, cell_value):
    """Count the number of game cells equal to either EMPTY, PEG, or INVALID"""
    total = 0
```



```
def count_cells(game, cell_value):
    """Count the number of game cells equal to either EMPTY, PEG, or INVALID"""
    total = 0
    for row in game.board:
        for cell in row:
            if cell == cell_value:
                total += 1
    return total


def count_playable_cells(game):
    """Playable cells are ones with a peg or empty hole"""
    return count_cells(game, "PEG") + count_cells(game, "EMPTY")


def make_board_with_peg_count(peg_count):
    """Makes English board size 9 with specific number of pegs, filling the first playable
spots, top to bottom"""
    layout = Board("English", 9)
    positions = []
    for row in range(9):
        for col in range(9):
            if layout.board[row][col] != "INVALID" and len(positions) < peg_count:
                positions.append((row, col))
    return make_custom_board(9, positions)
```

Show less

ChatGPT Answer to Prompt

Worked for 24s >

I've reviewed your Sprint 2 assignment, your user stories and acceptance criteria, and your `board.py`, `components.py`, and `peg_solitaire_gui.py` files.

The key distinction for your tests is that **Stories 4 and 5 should primarily be tested through the game logic in `Board`, not through Pygame's GUI**. That gives you genuine automated unit tests that can run without clicking anything or opening a game window. The GUI behavior, such as highlighting a selected peg and displaying the game-over message, can then be covered by manual tests.

Your Sprint 2 assignment explicitly requires at least two ChatGPT-generated unit tests, with screenshots of the prompts and responses and an explanation of errors you checked for or corrected. Writing tests for both stories should fit that requirement well.

One important detail from your implementation: `game_over_check()` returns `True` **when at least one legal move exists** and `False` when no legal moves remain. The name can be misleading, so we'll write the tests to match what the method actually does.

1. Automated unit tests for User Story 4: Make a move

Your `Board` class already contains the main logic needed for this story:

- `get_legal_moves(pos)` finds legal destinations for a peg.
- `is_valid_move(start_pos, end_pos)` checks whether a jump is legal.
- `make_move(start_pos, end_pos)` updates the board for a valid jump and returns `False` for an invalid one.

I'd recommend the following tests. They cover the core behavior without needing to launch Pygame.

Test code

Add these methods to your existing test class, or use the class below if you are organizing tests in a separate file. This assumes your helper functions are available in the same file.

Python

Run

```python
import unittest
from board import Board


class TestMakeMove(unittest.TestCase):

    def test_valid_move_updates_board_and_removes_one_peg(self):
        # Arrange: one legal horizontal jump
        game = make_custom_board(
            9,
            [(4, 2), (4, 3)]
        )
        initial_peg_count = game.count_pegs()

        # Act
        result = game.make_move((4, 2), (4, 4))

        # Assert
        self.assertTrue(result)
        self.assertEqual(game.board[4][2], "EMPTY")
```

Note: If your test file already imports `unittest`, `Board`, and defines the helper functions, do not duplicate those imports or helpers unnecessarily.

Why these tests are appropriate

Test	What it verifies	Related acceptance criterion
`test_valid_move_updates_board_and_removes_one_peg`	The starting hole and jumped peg become empty, the destination receives a peg, and the peg count decreases by one.	AC 4.2
`test_illegal_move_does_not_change_board`	An invalid jump is rejected without changing any board cell.	AC 4.3, logic portion
`test_legal_moves_returns_available_destination`	A peg with a legal jump has its destination identified correctly.	AC 4.1, logic portion

The third test verifies the legal-move calculation, not the visual selection or highlighting. Those GUI effects are handled separately.

2. Automated unit tests for User Story 5: A game is over

2. Automated unit tests for User Story 5: A game is over

Your implementation has three especially useful methods for this story:

- `game_over_check()` returns `True` if a legal move exists anywhere on the board, and `False` if no legal moves remain.
- `check_win()` returns `True` when exactly one peg remains.
- `get_rating()` returns the performance rating based on the remaining peg count.

These are all suitable for automated unit tests.

Test code

Python

Run

```python
class TestGameOver(unittest.TestCase):

    def test_game_continues_when_legal_moves_exist(self):
        # Arrange: a peg can jump over another peg
        game = make_custom_board(
            9,
            [(4, 2), (4, 3)]
        )

        # Act and assert
        self.assertTrue(game.game_over_check())
        self.assertFalse(game.check_win())

    def test_game_over_when_no_legal_moves_remain(self):
        # Arrange: one isolated peg cannot make a jump
        game = make_custom_board(
            9,
            [(0, 4)]
        )
```

Why these tests are appropriate

Test	What it verifies	Related acceptance criterion
`test_game_continues_when_legal_moves_exist`	The game is not ove ile a legal jump remains.	AC 5.3

Test	What it verifies	Related acceptance criterion
`test_game_continues_when_legal_moves_exist`	The game is not over while a legal jump remains.	AC 5.3
`test_game_over_when_no_legal_moves_remain`	The game-over check detects a board with no legal jumps.	AC 5.1, logic portion
`test_winning_game_with_one_peg_remaining`	A move leaving one peg is recognized as a win, with the correct rating.	AC 5.2, logic portion
`test_rating_matches_remaining_peg_count`	The rating matches the specified remaining peg counts.	AC 5.1 and AC 5.2, rating portion

The rating test uses your existing `make_board_with_peg_count()` helper. It does not require those pegs to be arranged in a playable position because this test only checks the rating calculation.

3. Manual tests for the GUI

Because your `PegSolitaireApp.game_screen()` method handles selection, highlighting, messages, and disabling input after the game ends, these behaviors are best checked manually with your current architecture.

The following are the manual tests I'd list in your Sprint 2 testing table.

User Story 4: Make a move

AC	Test input / action	Expected result
4.1	Click a peg with at least one legal jump.	Peg is highlighted yellow and its legal empty destinations are highlighted green.
4.2	Click a highlighted destination.	The peg moves, the jumped peg disappears, the original hole becomes empty, and the selection clears.
4.3	Select a peg and click an illegal destination.	Board remains unchanged, an error message appears, and the peg stays selected.
4.4	Click an empty hole, then click a peg with no legal moves.	No move occurs, nothing remains selected, and the appropriate message appears.
4.5	Select a peg, then click that same peg again.	The selection and destination highlights disappear without changing the board.
4.6	Select one movable peg, then click a different movable peg.	The second peg becomes selected, and only its legal destinations are highlighted.

User Story 5: A game is over

AC	Test input / action	Expe[illegible] result

User Story 5: A game is over

AC	Test input / action	Expected result
5.1	Play until no legal jumps remain, with more than one peg left.	`GAME OVER` appears, along with the remaining peg count and the correct rating.
5.2	Play until exactly one peg remains.	`YOU WON!` appears, and the Outstanding rating is displayed.
5.3	Make a move while another legal jump remains.	The game continues without a game-over message.
5.4	After the game ends, click a peg or empty hole.	The board remains unchanged and no move can be made.
5.5	After the game ends, click `RESTART`.	A fresh board of the same selected type and size appears, and the game-over message clears.

4. Two important implementation notes

First, keep the test classes together correctly. If you add both code snippets to the same test file, put `TestMakeMove` and `TestGameOver` above a single final block:

Python Run

```
if __name__ == "__main__":
    unittest.main()
```

Do not leave the first snippet's `unittest.main()` block between the two classes, because it can start test discovery before the second class has been defined.

Second, your GUI currently checks the end of the game automatically. In `game_screen()`, it checks `check_win()` first and then checks `game_over_check()`. This means your logic tests verify the conditions used by the GUI, but they do not independently prove that the correct message appears on screen. That is why the manual tests are important.

One further distinction: your assignment asks for at least two ChatGPT-generated unit tests, not necessarily one per user story. The tests above give you coverage for both stories, and you can document the two stories separately in your submission.

For the screenshots required by the assignment, capture your prompt and the generated tests, then run the tests yourself and document any corrections you make. I would specifically verify that the expected return values of `game_over_check()` match its actual implementation, since its name is easy to misinterpret.

Convert to PDF

I then prompted again to check to see if it met (supposedly) the requirements for this portion:

Convert to PDF

Alright, so overall its good right? I just need to make sure I put it in right and check for errors and make corrections if necessary. And it says at least 2, so there is no upper limit so if each method was actually more than one unit tests it should still meet the requirements correct?

Yes, **overall, you're on the right track.** Your plan to use multiple unit tests for User Stories 4 and 5 is reasonable, and having more than two tests should be fine.

Here is what I would make sure you do before submitting:

1. **Put the tests in the correct file.** Make sure the test file imports `Board` and includes or imports the helper functions, such as `make_custom_board()` and `make_board_with_peg_count()`.
2. **Run the tests yourself.** Don't assume the code is correct just because ChatGPT generated it. Check for import errors, incorrect board positions, and failed assertions.
3. **Correct any errors you find.** If a test fails, determine whether the test setup is wrong or your implementation has a bug. Document the corrections you make, as your assignment requests.
4. **Map the tests to the acceptance criteria.** Explain which criteria each test verifies. For example, a valid-move test supports Story 4's acceptance criterion about moving a peg and removing the jumped peg.
5. **Include the required evidence.** Save screenshots of your ChatGPT prompts and responses, and include your explanation of errors checked for and corrections made.

Does "at least 2" mean you can have more than two tests?

Yes. "At least 2" establishes a minimum, not a maximum. You can have multiple tests for each method, and that is generally good testing practice because different tests can check different behaviors and edge cases.

For example, you could have:

- `make_move()`: one test for a valid move and another for an invalid move.
- `game_over_check()`: one test when legal moves remain and another when no legal moves remain.
- `check_win()`: a test for one peg remaining.
- `get_rating()`: tests for the different rating categories.

That gives you several focused tests rather than only two.

One important distinction: the assignment's wording supports having at least two unit tests, but I would not assume that every method automatically needs its own test unless your instructor or rubric says so. Your current approach of covering multiple behaviors across Stories 4 and 5 is sensible.

The final step is to run everything against your actual project and confirm the tests pass. That is how you make sure the submission reflects your implementation rather than simply submitting generated code.

So I like the idea of the code you gave me, but could you please make it more organized, if possible, put a comment above each individual method test with the AC it corresponds to, and please make a method for each AC within the user story, does that make sense?

Worked for 15s >

Yes, that makes sense. I would organize the tests into **two test classes, one for User Story 4 and one for User Story 5**, with a separate test method for each acceptance criterion (AC). I'll put a comment immediately above each test method identifying the AC it covers.

One important detail: some acceptance criteria describe GUI behavior, such as highlighting a selected peg or displaying a message. Those need manual GUI tests unless your GUI is designed to support automated interaction tests. The automated tests below focus on the `Board` logic, while I'll identify the GUI-only criteria separately so you can account for every AC.

Here is the more organized version. It includes a separate test method for **every acceptance criterion, AC 4.1 through AC 4.6 and AC 5.1 through AC 5.5**. GUI-only tests are marked as skipped in the automated test file so you can verify them manually rather than pretending the `Board` class tests the GUI.

1. Organized unit test file

Create or update your test file, for example `test_peg_solitaire.py`. Keep your existing helper functions in this file or import them from wherever you defined them.

Python Run

```python
import unittest
from board import Board


class TestMakeMove(unittest.TestCase):
    """Tests for User Story 4: Make a move."""

    # AC 4.1: Selecting a peg that has a legal move.
    def test_ac_4_1_peg_has_legal_moves(self):
        game = make_custom_board(9, [(4, 2), (4, 3)])

        legal_moves = game.get_legal_moves((4, 2))

        self.assertIn((4, 4), legal_moves)

    # AC 4.2: Making a valid jump.
    def test_ac_4_2_valid_jump_updates_board(self):
        game = make_custom_board(9, [(4, 2), (4, 3)])
        initial_peg_count = game.count_pegs()
```

Why these tests are appropriate

Test	What it verifies	Related acceptance criterion
`test_game_continues_when_legal_moves_exist`	The game is not over while a legal jump remains.	AC 5.3
`test_game_over_when_no_legal_moves_remain`	The game-over check detects a board with no legal jumps.	AC 5.1, logic portion
`test_winning_game_with_one_peg_remaining`	A move leaving one peg is recognized as a win, with the correct rating.	AC 5.2, logic portion
`test_rating_matches_remaining_peg_count`	The rating matches the specified remaining peg counts.	AC 5.1 and AC 5.2, rating portion

The rating test uses your existing `make_board_with_peg_count()` helper. It does not require those pegs to be arranged in a playable position because this test only checks the rating calculation.

3. Manual tests for the GUI

Because your `PegSolitaireApp.game_screen()` method handles selection, highlighting, messages, and disabling input after the game ends, these behaviors are best checked manually with your current architecture.

The following are the manual tests I'd list in your Sprint 2 testing table.

User Story 4: Make a move

AC	Test input / action	Expected result
4.1	Click a peg with at least one legal jump.	Peg is highlighted yellow and its legal empty destinations are highlighted green.
4.2	Click a highlighted destination.	The peg moves, the jumped peg disappears, the original hole becomes empty, and the selection clears.
4.3	Select a peg and click an illegal destination.	Board remains unchanged, an error message appears, and the peg stays selected.
4.4	Click an empty hole, then click a peg with no legal moves.	No move occurs, nothing remains selected, and the appropriate message appears.
4.5	Select a peg, then click that same peg again.	The selection and destination highlights disappear without changing the board.
4.6	Select one movable peg, then click a different movable peg.	The second peg becomes selected, and only its legal destinations are highlighted.

2. What to check before running it

- Make sure `make_custom_board()` is defined or imported. The tests rely on it creating a board with the specified peg positions and all other playable positions empty.
- Your `Board` class uses `"PEG"` and `"EMPTY"` as its board values, which is why those exact strings appear in the assertions.
- `game_over_check()` returns `True` when at least one legal move remains, and `False` when no legal moves remain. The tests account for that behavior.
- The `@unittest.skip` methods are placeholders for GUI-only acceptance criteria. They will appear as skipped when you run the tests. **Do not count those as passing automated tests.** Complete their corresponding manual GUI checks instead.

Run the file with:

```bash
python -m unittest test_peg_solitaire.py -v
```

One more thing: the test for AC 5.1 uses two isolated pegs to create a no-legal-moves situation. Confirm that `(0, 4)` and `(4, 0)` are both playable positions on your English size-9 board. If your helper or board layout treats either position as invalid, replace them with two isolated pegs on valid positions.

3. Manual GUI checks for the skipped criteria

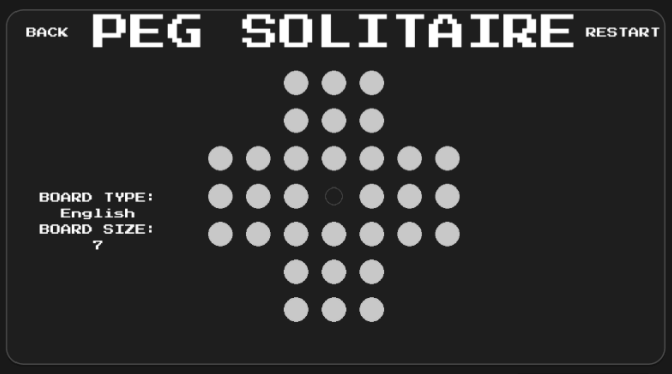
Acceptance criterion	What to verify manually
AC 4.5	Select a movable peg, click it again, and confirm the selection and highlights disappear.
AC 4.6	Select one movable peg, then another. Confirm only the second peg and its legal destinations are highlighted.
AC 5.4	End the game, click a peg or hole, and confirm the board does not change.
AC 5.5	After game over, start a new game and confirm the board resets and the message disappears.

Also manually verify the GUI messages and selection behavior for AC 4.1 through AC 4.4, and the displayed win/game-over message and restart option for AC 5.1 through AC 5.3. The automated `Board` tests cannot verify those visual behaviors by themselves.

This structure gives you clear method names, direct AC traceability for the Sprint 2 table, and a clean distinction between automated logic tests and manual GUI tests.

Convert to PDF

Prompt for Claude Portion:



import unittest from board import Board # Helper funcitons def make_custom_board(size, peg_positions, board_type="English"): """Creates a cust...

PASTED

Hello, I made this peg solitiare game. I personally struggle with being able to solve it, I want to make a unit test where it plays through a valid game. The rules are that orthogonal and diagonal pieces are allowed, within a 2 spot space jump. For example, lets say x x 0, where x are pegs and o is missing, after a move, it would be 0 0 x, as the peg left its spot and the middle one was removed. Same with diagonal. Could you give me a solution to this particular board? If you could put your answer in a python list format with ordered pairs, each row has 2 pairs, the first being the starting position coordinate and the second pair being the final position coordinate. The game is won

Show more

Ran a command ▸

I found a 31-move solution by search and replayed it in a simulator. Every move is a legal 2-space jump over a peg into an empty hole, and the final board has one peg left, at `(6, 3)`. It uses only orthogonal jumps, so it works whether or not your English board allows diagonals.

Paste this above the `# Test and and play a winning game` comment (or anywhere after `TestGameOver`):

python

```python
# Story 5 - full winning playthrough ======================
class TestWinningPlaythrough(unittest.TestCase):
    """Plays a complete, known-valid solution on English 7 and checks for a win."""

    # Each entry is (start (row, col), end (row, col)); 31 moves total
    ENGLISH_7_SOLUTION = [
        ((1, 3), (3, 3)),
        ((2, 1), (2, 3)),
        ((0, 2), (2, 2)),
        ((0, 4), (0, 2)),
        ((2, 3), (2, 1)),
        ((2, 0), (2, 2)),
        ((2, 4), (0, 4)),
        ((2, 6), (2, 4)),
        ((3, 2), (1, 2)),
        ((0, 2), (2, 2)),
        ((3, 0), (3, 2)),
        ((3, 2), (1, 2)),
        ((3, 4), (3, 2)),
        ((3, 6), (3, 4)),
        ((3, 4), (1, 4)),
        ((0, 4), (2, 4)),
        ((4, 2), (2, 2)),
        ((1, 2), (3, 2)),
        ((4, 0), (4, 2)),
        ((4, 3), (4, 1)),
        ((5, 4), (3, 4)),
        ((4, 6), (4, 4)),
        ((6, 2), (4, 2)),
        ((3, 2), (5, 2)),
        ((6, 4), (6, 2)),
        ((6, 2), (4, 2)),
        ((4, 1), (4, 3)),
        ((4, 3), (4, 5)),
        ((2, 4), (4, 4)),
        ((4, 5), (4, 3)),
        ((4, 3), (6, 3)),
    ]

    def test_english_7_full_winning_game(self):
        game = Board("English", 7)
        self.assertEqual(game.count_pegs(), 32)

        for move_number, (start, end) in enumerate(self.ENGLISH_7_SOLUTION, 1):
            with self.subTest(move=move_number, start=start, end=end):
                self.assertTrue(
```

Updated User Stories:

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inPprogress)
1. Choose a board size	1.1	AC 1.1 Successful pick of a valid board size **Given**: The game board setup screen is displayed with a default board size of 7 **When**: I select an odd whole number between 5 and 9 (5, 7, 9), (e.g: 7) in the board size field **Then**: The board size field shows 7 (or whatever the selected size was) **And** the game will be created with a board size of 7	Completed
	1.2	A.C 1.2 Restriction of Even Board Sizes Given: The game boardsetup screen is displayed When: The userviews the options for the board size Then: The only valid options are 5, 7, and 9 as radio buttons And: Even numbers such as 6 or 8 are excluded from the list	Completed
	1.3	AC 1.3 Rejecting Against Invalid or Out of Range inputs for board size **Given:** The Game board setup screen is displayed **When:** The user interacts with the board size selection **Then:** The UI restricts input strictly through the provided radio buttons, preventing out of range values, decimals, and symbols from being chosen	Completed
2. Choose the board type	2.1	AC 2.1 Default board type and size upon launch of game **Given:** The game has just been launched **When:** The game setup screen is displayed **Then:** The three main board types are offered to pick from (English, Hexagon, Diamond) **And** English is selected by default	Completed
	2.2	AC 2.2 Successful Selection of game board type **Given:** the setup screen is displayed and English is selected (for example) **When:** I select a different board type (example, Hexagon) **Then:** Hexagon becomes the selected board type **And** English is no longer selected **And** The next game I play with be created with the Hexagon board type (applies to other board types, just Hexagon for this scenario)	Completed
	2.3	AC 2.3 Only one board type can be selected at a time **Given:** the setup screen is selected and Diamond is the chosen board type (for example, will apply to other ones) **When:** I select Hexagon (or any other board type) **Then:** only Hexagon is selected (or other chosen board type) **And** the current board size remains unchanged	Completed
3. Start a new game of the chosen board size and type	3.1	AC 3.1 Starting a new English game **Given:** the board size is 7 (default) and the game is set to English **When:** a new game is created by clicking the button **Then:** a cross shaped board with 33 positions are shown And 32 contain a peg And only middle peg space is empty	Completed
	3.2	AC 3.2 Creating a game of a different board size and type **Given:** the board type is Diamond and size is 9 (for example) **When:** I click the button to create a new game **Then:** A diamond shaped board with 41 peg positions are shown And 40 contain a peg (for this specific example) And only the center peg space is empty	Completed

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inPprogress)
	3.3 (No longer applicable, covered under stories 1 and 2)	AC 3.3 Rejecting the ability to start a new game of invalid board size (out of range, even, non numeric character) **Given:** The board field contains an invalid value (as mentioned in header) **When:** I click the button to create a new game **Then:** An error message is displayed (no more error message) And the previous board type remains selected And the board size defaults to 7 Note: This will be removed, but is simply staying here as a reminder.	Complete (no longer applicable as own acceptance criteria as it is too similar to acceptance criteria in stories 1 and 2 to remove redundancy
4. Make a move in a game	4.1	**AC 4.1: Selecting a peg that has a legal move** **Given**: A game is in progress and the board is displayed **When**: I click a peg that has at least one legal jump available **Then**: The peg is shown as selected **And**: Each empty hole it can legally jump into is highlighted	Completed
	4.2	**AC 4.2: Making a valid jump** **Given**: A peg is selected and a highlighted empty hole is two positions away in a straight line (either diagonal or orthogonal), with a peg between them **When**: I click that empty hole **Then**: The selected peg moves into the empty hole **And**: The peg that was jumped over is removed from the board **And**: The hole the peg started in becomes empty **And**: The total number of pegs on the board decreases by one	Completed
	4.3	**AC 4.3: Rejecting an illegal destination** **Given**: A peg is selected **When**: I click a hole or position that is not a legal jump **Then**: No move is made and the board stays unchanged **And**: A message is displayed indicating that the move is not allowed **And**: The peg becomes unselected	Completed
	4.4	**AC 4.4: Rejecting selection of an empty hole or a peg with no legal moves** **Given**: A game is in progress **When**: I click an empty hole as the starting point, or a peg that has no legal jumps **Then**: Nothing is selected and the board stays unchanged **And**: A message is displayed indicating that there is no move available from that position	Completed
	4.5	**AC 4.5: Deselecting a peg** **Given**: A peg is selected **When**: I click the selected peg again **Then**: The peg is no longer selected **And**: The destination highlights disappear **And**: The board stays unchanged	Completed
	4.6	**AC 4.6: Selecting another legal peg when one is currently selected** **Given:** A peg is selected	Completed

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inPprogress)
		When: I click another peg with a legal move **Then:** The new peg is highlighted along with its legal moves And: The previously peg that was selected no longer shows it as selected And: The previous peg's legal moves are no longer shown	
5. A game is over	5.1	**AC 5.1: Game ends when no legal moves remain** Given: A game is in progress and more than one peg remains on the board When: I make a move that leaves no peg with a legal jump Then: A message is displayed saying the game is over because no legal moves remain And: The number of pegs remaining is displayed And: I can select an option to create a new game (either restart current one to play the same mode, or press back button to select a different one) And: I can see my game performance, Very Good (2 marbles left), Good (3 marbles left), Average (4 or more left).)	Completed
	5.2	**AC 5.2: Winning the game** Given: A game is in progress When: I make a move that leaves exactly one peg on the board Then: A message is displayed saying that I won And: The final peg count of 1 is shown And: I can select an option to create a new game And: I can see my rating of Outstanding (1 marble left)	Completed
	5.3	**AC 5.3: Game continues while legal moves exist** Given: A game is in progress When: I make a move and at least one peg still has a legal jump Then: The game does not end And: No game over message is displayed	Completed
	5.4	**AC 5.4: Moves are disabled after the game ends** Given: The game over message is displayed When: I click a peg or an empty hole on the board Then: No peg is selected and no move is made And: The board stays as it was when the game ended And I can see my game rating (Outstanding (1 marbleleft), Very Good (2 marbles left), Good (3 marbles left), Average (4 or more left).	Completed
	5.5	**AC 5.5: Starting a new game after game over** **Given: The game over message is displayed** **When: I click the button to start a new game (via restart or going back)** **Then: A new board is created using the selected board type and size** **And: The game over message is cleared** **And: I can play the new game that was created (via restart or back)**	Completed
6. Enabling/Disabling Music	6.1	AC 6.1 Default playing of game music **Given:** The game has been launched **When:** I see the opening screen **Then:** I should be able to hear the provided game music And a button with a speaker icon indicating that music is enabled appears	toDo
	6.2	AC 6.2: Turning off the music **Given:** The game music is playing **When:** The toggle button has the icon to show that music is playing	toDo

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inPprogress)
		Then: The music should turn off when the button is clicked And: The toggle button icon should display an X on it, indicating it is off	
	6.3	AC 6.3: Turning on the music **Given**: The game music is off **When**: the toggle button has the icon to show that music is not playing **Then:** The music plays And it starts at the beginning of the soundtrack	toDo
	6.4	AC 6.4: The soundtrack ends **Given**: The game music is playing **When:** The length of the soundtrack finishes **Then**: The music soundtrack should restart from the beginning (not be silent)	toDo
7. Choose a theme	7.1	AC 7.1: Default theme is shown upon launch **Given:** The game is launched **When:** The game opening screen is displayed **Then:** The default theme should be displayed The default theme button should be selected And the other themes should not be selected	toDo
	7.2	AC 7.2: Changing theme **Given:** The game is open in any screen **When:** The button for a different theme than the current one is clicked **Then:** The background theme should change to the corresponding selected them And the button should be shown as selected And the previous theme button should not be selected And other theme buttons should not be selected	toDo
8. Give move hints	8.1	AC 8.1: Clicking hints button to show hints during gameplay **Given:** A game has started and the board is displayed **When:** I click the button to show hints **Then:** A colored ring will appear around each and every peg that has a legal move And: There will be a message displaying and number of how many pegs can be moved	toDo
	8.2	AC 8.2: Clicking hints when there are no more legal moves **Given:** A game has started and the board is displayed **When:** I click the hint button and there are no more legal moves **Then:** A message is displayed saying that there are no more legal moves and that this game is over	toDo
9. Play the game in Autoplay	9.1	AC 9.1: Starting autoplay **Given:** a game is in progress and there is at least one legal move **When:** I click the autoplay button **Then:** The computer makes moves on the board without user input **And** each move is shown on the board as it is made (pegs jump and disappears and moving peg lands in empty hole as appropriate)	toDo
	9.2	AC 9.2: Autoplay moves are randomized and legal **Given:** Autoplay is running **When:** the computer chooses its next move **Then:** The move is a legal move (orthogonal or diagonal over adjacent peg into empty hole two positions away - description in project overview) And: The move is picked at random from all available legal moves (not hardcoded)	toDo

User Story ID and Name	AC ID	Description of Acceptance Criterion	Status (completed, toDo, inPprogress)
	9.3	AC 9.3: Autoplay ends with no legal moves remaining **Given:** Autoplay is running **When:** a move leaves with no more legal moves available **Then:** Autoplay stops with a game over message displayed And the number of pegs remaining is shown And the performance rating is displayed (Good, Avg. etc.) And I can select an option to create a new game	toDo
	9.4	AC 9.4: Manual input is blocked/not allowed while autoplay is on **Given:** Autoplay is running **When:** I click a peg or empty hole on the board **Then:** No peg is selected and no move is played And autoplay continues as normal (unaffected)	toDo
	9.5	AC 9.5: Autoplay works on every board type and size **Given:** A new game is created with the supported/available board types (English, hexagon, Diamond) and valid board size (5,7,9) **When:** I click to enable autoplay **Then:** Autoplay plays the game to completion following the same rules as manual play would on the board	toDo
	9.6	AC 9.6: Starting a new game after autoplay **Given:** An autoplay game has ended with game over message **When:** I click o start a new game **Then:** A new board is created with designated type and size And The game over message is cleared And I can play manually or in autoplay	toDo

Submission Guidelines:

Each student must independently upload **LastName_FirstName_Sprint2.docx** to Canvas. Include:

1. **Your name**
2. **GitHub repository link** containing all production code and automated tests. Ensure that the link is public.
3. **Demonstration video link** — a narrated video of no more than 3 minutes showing the required features and automated tests (again the link should be public).
4. **Source-code summary table** — filenames, production/test classification, line counts, and total.
5. **Updated user stories and acceptance criteria**, mapped to implementation classes and methods, with completion status.
6. **Testing tables** — automated test mappings and results, plus manual or additional tests where applicable.
7. **LLM evidence** — screenshots of prompts and responses used to generate at least two unit tests, with an explanation of how you checked them and corrected any errors.

Mark unused optional sections **"Not applicable."** Before submitting, verify that the instructor can access your GitHub repository and video.